Optimizing a 3D Data Cube Volume Render

GuanLong Zhou, Theodor Fritsch, Linus Rackner

March 2026

1 Introduction

The purpose of the code that we will analyze and optimize is to render a three dimensional data cube. This datacube contains density data which the code transforms to color values which are then saved to an image. The code contains several parts that we want to profile and hopefully optimize. The original code was taken from here: github.

2 Methodology

2.1 Hypothesis

The project started with a hypothesis of which part of the existing, original code was the bottleneck. By looking at the code the largest bottleneck appears to be the transfer function since it has to perform several operations on a large list.

2.2 Profiling Method

Firstly, *cProfile* was used to identify, on a high level, time-consuming functions and parts. The cumulative time was extracted, and used to identify function-bottlenecks.

Functions that were identified as the slowest were analysed using the `line_profiler`, to understand which specific line/lines were the reason for the slow behaviour.

We deemed that only these two profilers were necessary to find bottlenecks.

2.3 Optimization Method

Optimization was based on profiling results. Alternative implementations of bottlenecks were tested, and after each modification, the profiling process was repeated to evaluate the improved implementations. Optimization methods were inspired by lectures in the course, from *Module II: Data Structures and Methods for HPC* and *Module III: Using Compilation Techniques and GPUs for Performance Optimization*.

2.4 Validation Method

The program is deterministic and will return the same output for identical inputs. Ensuring functional correctness of altered, hopefully optimized, versions was verified by comparing their outputs against the original implementation's output, for the same input-data.

Performance improvements were measured through benchmarking. Each implementation was executed under the same conditions, and were averaged to ensure more reliable comparison.

3 Experimental Setup

The code was executed on the following machine in a linux enviroment: Intel i7 14th gen 14700F (cpu), NVIDIA RTX 4070 (gpu) and 64GB RAM. To profile the code we used cProfile in order to measure execution time for the program and its functions, line_profiler in order to understand what takes down down to single lines in the program and the python time module in order to manually measure the execution time of certain parts of the program.

3.1 Creating Different Sized Data Sets

To check how performance scales with different problem sizes, additional artificial datasets were created. These datasets originate from the original, given data set *datacube.hdf5*. They were "refactored" with the factors $factor \in \{1.5, 2.0, 2.5, 3.0\}$. Larger datasets were tested, but often demanded too much memory. The actual code used can be found in the Appendix B. See listing 1 for a short pseudo-code version.

```
# Load HDF5
let f = read_file("datacube.hdf5") # original data-file
let data = f["density"] # shape = (256, 256, 256)
let original_size = data.shape[0]

let FACTORS = [1.5, 2.0, 2.5, 3.0]
for FACTOR in FACTORS:
    let SIZE = FACTOR * original_size

    let new_data = zoom(data, FACTOR, order=3)

    [Write to new file: "datacube_{SIZE}.hdf5"]
```

Listing 1: "Pseudo-code" for creating artifical datasets

4 Profiling Results

4.1 cProfile

The results from the high-level profiler `cProfile` were cluttered since library-functions such as `SciPy`'s `interpn()` or `NumPy`'s `meshgrid()` themselves call other functions. To improve readability, the results were sorted by cumulative time, and only non-private functions are present in Table 5. The original output from `cProfile`, including the 15 most expensive function-calls, can be found in Appendix D.1.

Function	Calls	Total Time (s)	Cumulative Time (s)
`main()`	1	0.887	17.451
`interpn()`	10	0.177	12.833
`transferFunction()`	1800	3.361	3.361

Table 1: Top functions ranked by cumulative runtime from `cProfile`.

4.2 Line profiler

The results from `line_profiler` gave two primary bottlenecks. The most significant, taking **69.7%** of the total execution time, was the call to `interpn()`. `interpn()` performs a multidimensional interpolation, and is a part of the `SciPy` library. The call to `interpn()` is on row *73*.

The second bottleneck identified was the function `transferFunction()`, which accounted for **22.3%** of the total execution time, called on row *82*. Although less dominant than the interpolation step, the repeated calls to this function within the loop still contributed significantly to the overall runtime. See Table 2 for the most time-consuming operations. See Appendix D.2 for the full output from the `line_profiler`.

Line	Hits	Time (ms)	Per Hit (ms)	% Time
66	50	833.9	16.7	1.2
68	50	874.7	17.5	1.3
69	50	913.8	18.3	1.4
70	50	856.4	17.1	1.3
73	50	46837.3	936.7	69.7
82	9000	15004.1	1.7	22.3
83	9000	618.9	0.1	0.9
84	9000	631.4	0.1	0.9
85	9000	623.7	0.1	0.9

Table 2: Profiling results obtained using `line_profiler`. Only the most time-consuming rows are shown.

5 Results

The two bottlenecks identified were `interpn` and `transferFunction`. These bottlenecks can be optimized separately, profiled and benchmarked and then combined for a optimized version, see 4

5.1 Optimization of `interpn`

When changing all calls to the numpy library with calls to CuPy, which executes on the GPU we see an approximate doubling in execution time compared to running the code as it came from github.

Interestingly the time distribution is slightly changed when you check which line of code takes the most time. In the original code the row wqhich called the `interpn` to 70% of the time but in the GPU optimized version which uses cupy's interpn variant it only consumes 8.2% of the total time while the call to the function `transferFunction` consumes 72% of the execution time. To see the line profiling look at G.

5.2 Optimization of `transferFunction`

To allow testing focused on only `transferFunction`, a fake data set can be created in order to avoid calling `interpn`. Furthermore, the new data set allows measuring one single call to `transferFunction`. See table 3

```
fake_dataslice = np.random.rand(N, N)
transferFunction(np.log(fake_dataslice))
```

Listing 2: Creating and benchmarking `transferFucntion` with a fake data set

5.2.1 Results from benchmark_transferfunc.py

Table 3 shows the mean run time and the relative to the original implementation speedup.

Implementation	Mean Time (ms)	Std (ms)	Speedup
Original	1.439	0.081	1.00×
Cython	1.717	0.120	0.84×
NumExpr	2.005	0.174	0.72×
Inplace	1.450	0.177	0.99×
CuPy	0.948	0.102	1.52×

Table 3: Benchmark results for different implementations of `transferFunction()`. Speedup is relative to the original implementation.

5.3 Performance

5.3.1 Execution Time

Version	256	384	512	640	768
Original	12.115	12.290	12.353	12.512	12.882
CuPy	2.416	2.683	2.799	2.499	2.510

Table 4: Comparing the Original version with cupy implemented version, in seconds for an average of 20 iterations on each size.

5.4 Profiling the Optimized CuPy version

Function	**Calls**	**Total Time (s)**	**Cumulative Time (s)**
`main()`	1	0.336	2.621
`transferFunction()`	1800	1.688	1.691

Table 5: Top functions ranked by cumulative runtime from `cProfile`.

6 Discussion and Conclusion

6.1 Execution Time

As shown in table 4, the run times of both the original and optimized version seems to not be affected that much by the grid-sizes of the data in the hdf5 file. This caught us by surprise, as we had thought that this would affect the performance quite a lot since the number of datapoints is linear to the size of the cube.

A plausible reason to why the original version was barely affected by the grid-size, could be the back-end `SciPy`-implementation of `interpn`, which could support vectorized evalutation. By operating on entire arrays of data, the function could rely on low-level function rather than high-level loops, allowing better cache utilization. This, in combination with ”modern” hardware could be the explanation to as why the increases in grid-sizes had a smaller effect on run-time than predicted.

For the `CuPy` version, i.e. the program ported to the GPU, a possible reason to why the performance was not marginaly affected by the grid size, could be that the problem size was never large enough to fully utilize the GPU's memory.

6.2 `CuPy` Optimization

Optimizing with `CuPy` and executing the program in parallel on the GPU gave very promising results in the form of a large speedup in total execution time. This was to be expected since the program relied heavily on numpy array operations which `CuPy` is very good at translating to parallel GPU operations.

6.3 `transferFunction` Optimization

As show in table 3, the only speed-up implementation is the **CuPy**-version. The second fastest version was the original implementation. To our surprise, the in-place implementation did not improve run-time. A plausible reason of this could be that the initialization of the array led to overhead, perhaps when allocating memory, while the out-of-place implementation did not require an array of all four *rgba*-values. Similar to the in-place implementation, the `numexpr` version likely led to overhead when evaluating the expressions.

Furthermore, we did not notice any improvement in execution time when optimizing the `tranferFunction` using Cython, this probably depends on the fact that the function requires a lot of interaction with the python virtual machine because of the many calls to the numpy library. One possible future approach could be to rewrite the numpy functions from scratch in order to fully take advantage of the Cython optimizations but this was deemed to time consuming for this project and beyond our scope.

6.4 Final Optimization

As table 3 shows, the most significant (and only) speed-up of `transferFunction` was made with the **CuPy**-implementation. Hence, the final, most optimized version, is a pure **CuPy**-implementation.

6.5 Conclusion

In conclusion we have found that this program is best suited to be optimized with parallel computing and in this case running it on the GPU. The nature of the volume rendering algorithm involves a large number of operations on multi-dimensional arrays, many of which can be performed independently. This type of workload maps well to the massively parallel architecture of modern GPUs, where thousands of threads can execute similar operations simultaneously. By offloading the computationally intensive parts of the program to the GPU using CuPy, the execution time was significantly reduced compared to the original CPU implementation.

Further studies could be done on how the program performs when run in parallel on the CPU using for example Python's multiprocessing library. Such an approach could allow the workload to be distributed across multiple CPU cores, potentially improving performance on systems where a GPU is not available. Investigating other CPU-based parallelization techniques, such as vectorized implementations or just-in-time compilation approaches, could also provide further insights into how the algorithm behaves on different hardware architectures.

Regarding why the other approaches did not succeed in improving performance, this likely depends on the fact that NumPy, which the program heavily relies on, is already a highly optimized library designed for high-performance numerical computations. Many NumPy operations are implemented in low-level languages such as C and make use of optimized mathematical libraries.

As a result, attempts to accelerate the code using techniques such as Cython did not produce noticeable improvements, since the performance-critical operations were already executed efficiently at the library level. This illustrates that when a program already relies heavily on optimized numerical libraries, further speedups are often better achieved by changing the execution model, for example by introducing parallel computation on GPUs rather than attempting to optimize individual Python-level operations.

Overall, the results of this project demonstrate the importance of selecting an optimization strategy that matches the computational characteristics of the problem. In this case, GPU-based parallelism proved to be the most effective approach for improving the performance of the volume rendering pipeline.

Appendix A Original Code

Original code:

```
import numpy as np
import matplotlib.pyplot as plt
import h5py as h5
from scipy.interpolate import interpn

"""
Create Your Own Volume Rendering (With Python)
Philip Mocz (2020) Princeton University, @PMocz

Simulate the Schrodinger-Poisson system with the Spectral method
"""


def transferFunction(x):
    r = (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    g = (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 1.0 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    b = (
        0.1 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 1.0 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    a = (
        0.6 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.01 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )

    return r, g, b, a


def main():
    """Volume Rendering"""
```

```
# Load Datacube
f = h5.File("datacube.hdf5", "r")
datacube = np.array(f["density"])

# Datacube Grid
Nx, Ny, Nz = datacube.shape
x = np.linspace(-Nx / 2, Nx / 2, Nx)
y = np.linspace(-Ny / 2, Ny / 2, Ny)
z = np.linspace(-Nz / 2, Nz / 2, Nz)
points = (x, y, z)

# Do Volume Rendering at Different Viewing Angles
Nangles = 10
for i in range(Nangles):
    print("Rendering Scene " + str(i + 1) + " of " + str(
        Nangles) + ".\n")

    # Camera Grid / Query Points -- rotate camera view
    angle = np.pi / 2 * i / Nangles
    N = 180
    c = np.linspace(-N / 2, N / 2, N)
    qx, qy, qz = np.meshgrid(c, c, c)
    qxR = qx
    qyR = qy * np.cos(angle) - qz * np.sin(angle)
    qzR = qy * np.sin(angle) + qz * np.cos(angle)
    qi = np.array([qxR.ravel(), qyR.ravel(), qzR.ravel()]).T

    # Interpolate onto Camera Grid
    camera_grid = interpn(points, datacube, qi, method="linear
        ").reshape((N, N, N))

    # Do Volume Rendering
    image = np.zeros((camera_grid.shape[1], camera_grid.shape
        [2], 3))

    for dataslice in camera_grid:
        r, g, b, a = transferFunction(np.log(dataslice))
        image[:, :, 0] = a * r + (1 - a) * image[:, :, 0]
        image[:, :, 1] = a * g + (1 - a) * image[:, :, 1]
        image[:, :, 2] = a * b + (1 - a) * image[:, :, 2]

    image = np.clip(image, 0.0, 1.0)

    # Plot Volume Rendering
    plt.figure(figsize=(4, 4), dpi=80)
```

```
        plt.imshow(image)
        plt.axis("off")

        # Save figure
        plt.savefig(
            "volumerender" + str(i) + ".png", dpi=240, bbox_inches
                ="tight", pad_inches=0
        )

    # Plot Simple Projection -- for Comparison
    plt.figure(figsize=(4, 4), dpi=80)

    plt.imshow(np.log(np.mean(datacube, 0)), cmap="viridis")
    plt.clim(-5, 5)
    plt.axis("off")

    # Save figure
    plt.savefig("projection.png", dpi=240, bbox_inches="tight",
        pad_inches=0)
    plt.show()

    return 0

if __name__ == "__main__":
    main()
```

Appendix B Creating Artificial Datasets

The following code was used to create artificial datasets, varying in sizes.

```
import numpy as np
import matplotlib.pyplot as plt
import h5py
from scipy.ndimage import zoom

# Load HDF5
with h5py.File("datacube.hdf5", "r") as f:
    data = f["density"][:] # shape (256,256,256)

data_384 = zoom(data, 1.5, order=3)

data_512 = zoom(data, 2.0, order=3)

data_640 = zoom(data, 2.5, order=3)
```

```
data_768 = zoom(data, 3.0, order=3)

with h5py.File(f'data_384.hdf5', 'w') as f:
    dset = f.create_dataset("density", data = data_384)
    print(f"Done with size :{dset.shape}")

with h5py.File(f'data_512.hdf5', 'w') as f:
    dset = f.create_dataset("density", data = data_512)
    print(f"Done with size :{dset.shape}")

with h5py.File(f'data_640.hdf5', 'w') as f:
    dset = f.create_dataset("density", data = data_640)
    print(f"Done with size :{dset.shape}")

with h5py.File(f'data_768.hdf5', 'w') as f:
    dset = f.create_dataset("density", data = data_768)
    print(f"Done with size :{dset.shape}")
```

Appendix C `transferFunction` Benchmarking

All the functions were imported. `benchmark_function` was run for the original, in-place, numexpr and Cython implementation.

```
iterations = 100

def benchmark_function(func, log_input):
    times = []

    # Warmup
    for _ in range(5):
        func(log_input)

    for _ in range(iterations):
        t1 = time.perf_counter()
        func(log_input)
        t2 = time.perf_counter()
        times.append(t2 - t1)

    return statistics.mean(times), statistics.stdev(times)
```

C.1 In-Place Implementation

In-place implementation of `transferFunction`.

```
def transferFunction(x, rgba):
    rgba[0] += (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    rgba[1] += (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 1.0 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    rgba[2] += (
        0.1 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 1.0 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    rgba[3] += (
        0.6 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.01 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )

...
# in main():

    # initialize rgba before calling transferFunction
    rgba = np.zeros((4,) + dataslice.shape)
    transferFunction(np.log(dataslice), rgba)
```

C.2 `numexpr`

`numexpr` implementation of `transferFunction`.

```
def transferFunction(x, rgba):
    e1 = np.exp(-((x - 9.0) ** 2) / 1.0)
    e2 = np.exp(-((x - 3.0) ** 2) / 0.1)
    e3 = np.exp(-((x - -3.0) ** 2) / 0.5)

    evaluate("1.0 * e1 + 0.1 * e2 + 0.1 * e3", out=rgba[0])
    evaluate("1.0 * e1 + 1.0 * e2 + 0.1 * e3", out=rgba[1])
    evaluate("0.1 * e1 + 0.1 * e2 + 1.0 * e3", out=rgba[2])
    evaluate("0.6 * e1 + 0.1 * e2 + 0.01 * e3", out=rgba[3])

    return rgba[0], rgba[1], rgba[2], rgba[3]
```

```
...
# in main():

    # initialize rgba before calling transferFunction
    rgba = np.zeros((4,) + dataslice.shape)
    transferFunction(np.log(dataslice), rgba)
```

C.3 cythonfn.pyx

cython implementation of transferFunction. The following listing is the code in cythonfn.pyx

```
# cython: boundscheck=False
# cython: wraparound=False
# cython: cdivision=True

import numpy as np
cimport numpy as np
from libc.math cimport exp

def cython_transferFunction(np.ndarray[np.double_t, ndim=2] x):

    cdef np.ndarray[np.double_t, ndim=2] r = np.empty((x.shape[0],
         x.shape[1]), dtype=np.float64)
    cdef np.ndarray[np.double_t, ndim=2] g = np.empty((x.shape[0],
         x.shape[1]), dtype=np.float64)
    cdef np.ndarray[np.double_t, ndim=2] b = np.empty((x.shape[0],
         x.shape[1]), dtype=np.float64)
    cdef np.ndarray[np.double_t, ndim=2] a = np.empty((x.shape[0],
         x.shape[1]), dtype=np.float64)

    r = (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    g = (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 1.0 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    b = (
        0.1 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 1.0 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
```

```
    a = (
        0.6 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.01 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )

    return r,g,b,a
```

C.4 `setup.py`

`setup.py` in order to enable usage of `pythonfn`.

```
from distutils.core import setup
from Cython.Build import cythonize
import numpy

setup(ext_modules=cythonize("cythonfn.pyx",
                            compiler_directives={"language_level":
                                "3"}),
                            include_dirs=[numpy.get_include()])
```

Appendix D Profiling-Appendix

D.1 Original `cProfile` Results

```
$ python3 -m cProfile -s cumulative volumerender.py
True
         2016437 function calls (1977858 primitive calls) in
             24.429 seconds

   Ordered by: cumulative time

   ncalls tottime percall cumtime percall filename:lineno(function
       )
    710/1 0.008 0.000 24.441 24.441 {built-in method builtins.exec
        }
        1 0.003 0.003 24.441 24.441 volumerender.py:1(<module>)
        1 0.887 0.887 17.451 17.451 volumerender.py:42(main)
       10 0.177 0.018 12.833 1.283 _rgi.py:645(interpn)
       90 0.008 0.000 12.824 0.142 __init__.py:1(<module>)
       10 0.026 0.003 12.632 1.263 _rgi.py:375(__call__)
       10 10.978 1.098 10.980 1.098 _rgi.py:520(_evaluate_linear)
    865/4 0.014 0.000 6.980 1.745 <frozen importlib._bootstrap
        >:1349(_find_and_load)
```

```
   853/4 0.012 0.000 6.979 1.745 <frozen importlib._bootstrap
       >:1304(_find_and_load_unlocked)
   826/6 0.010 0.000 6.975 1.163 <frozen importlib._bootstrap
       >:911(_load_unlocked)
  2064/10 0.008 0.000 6.975 0.697 <frozen importlib._bootstrap
      >:480(_call_with_frames_removed)
   690/6 0.004 0.000 6.974 1.162 <frozen importlib.
       _bootstrap_external>:1017(exec_module)
  629/50 0.002 0.000 5.745 0.115 {built-in method builtins.
      __import__}
 734/135 0.010 0.000 5.474 0.041 <frozen importlib._bootstrap
      >:1390(_handle_fromlist)
    1800 3.361 0.002 3.361 0.002 volumerender.py:16(
        transferFunction)
```

D.2 line_profiler_results

```
Timer unit: 1e-06 s

Total time: 67.2385 s
File: volumerender.py
Function: main at line 39

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    39                                           @profile
    40                                           def main():
    41                                               """Volume Rendering
    """
    42
    43                                               # Load Datacube
    44         1        429.2    429.2      0.0      f = h5.File("
    datacube.hdf5", "r")
    45         1      13976.9  13976.9      0.0      datacube = np.array(
    f["density"])
    46
    47                                               # Datacube Grid
    48         1          2.1      2.1      0.0      Nx, Ny, Nz =
    datacube.shape
    49         1         90.3     90.3      0.0      x = np.linspace(-Nx
    / 2, Nx / 2, Nx)
    50         1         22.8     22.8      0.0      y = np.linspace(-Ny
    / 2, Ny / 2, Ny)
    51         1         15.7     15.7      0.0      z = np.linspace(-Nz
    / 2, Nz / 2, Nz)
    52         1          0.3      0.3      0.0      points = (x, y, z)
    53
```

```
54                                                  # Store images for
GIF creation
55         1          0.4      0.4      0.0        images = []
56
57                                                  # Do Volume
Rendering at Different Viewing Angles
58         1          0.2      0.2      0.0        Nangles = 50
59        51         21.0      0.4      0.0        for i in range(
Nangles):
60                                                      # print("
Rendering Scene ...")
61
62                                                      # Camera Grid /
Query Points
63        50        110.6      2.2      0.0            angle = np.pi /
2 * i / Nangles
64        50          9.6      0.2      0.0            N = 180
65        50       3068.9     61.4      0.0            c = np.linspace
(-N / 2, N / 2, N)
66        50     833900.8  16678.0      1.2            qx, qy, qz = np.
meshgrid(c, c, c)
67        50       7646.8    152.9      0.0            qxR = qx
68        50     874672.5  17493.5      1.3            qyR = qy * np.
cos(angle) - qz * np.sin(angle)
69        50     913779.9  18275.6      1.4            qzR = qy * np.
sin(angle) + qz * np.cos(angle)
70        50     856408.8  17128.2      1.3            qi = np.array([
qxR.ravel(), qyR.ravel(), qzR.ravel()]).T
71
72                                                      # Interpolate
onto Camera Grid
73        50   46837295.7 936745.9     69.7            camera_grid =
interpn(points, datacube, qi,
74
                method="linear"
75
            ).reshape((N, N, N))
76
77                                                      # Do Volume
Rendering
78        50       2611.7     52.2      0.0            image = np.zeros
((camera_grid.shape[1],
79
            camera_grid.shape[2], 3))
80
81      9050      13089.2      1.4      0.0            for dataslice in
 camera_grid:
82      9000   15004075.6   1667.1     22.3                r, g, b, a =
 transferFunction(np.log(dataslice))
```

```
    83      9000     618876.3     68.8      0.9              image[:, :,
    0] = a * r + (1 - a) * image[:, :, 0]
    84      9000     631440.5     70.2      0.9              image[:, :,
    1] = a * g + (1 - a) * image[:, :, 1]
    85      9000     623652.9     69.3      0.9              image[:, :,
    2] = a * b + (1 - a) * image[:, :, 2]
    86
    87        50       3230.1     64.6      0.0          image = np.clip(
    image, 0.0, 1.0)
```

Appendix E Benchmarking code

Code for benchmarking/getting performance of original code:

```
import numpy as np
import matplotlib.pyplot as plt
import h5py as h5
from scipy.interpolate import interpn
import time
import statistics

"""
Create Your Own Volume Rendering (With Python)
Philip Mocz (2020) Princeton University, @PMocz

Simulate the Schrodinger-Poisson system with the Spectral method
"""


def transferFunction(x):
    r = (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    g = (
        1.0 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 1.0 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    b = (
        0.1 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 1.0 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )
    a = (
```

```
        0.6 * np.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * np.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.01 * np.exp(-((x - -3.0) ** 2) / 0.5)
    )

    return r, g, b, a


def main(path):
    """Volume Rendering"""

    # Load Datacube
    f = h5.File(path, "r")
    datacube = np.array(f["density"])

    # Datacube Grid
    Nx, Ny, Nz = datacube.shape
    x = np.linspace(-Nx / 2, Nx / 2, Nx)
    y = np.linspace(-Ny / 2, Ny / 2, Ny)
    z = np.linspace(-Nz / 2, Nz / 2, Nz)
    points = (x, y, z)

    # Do Volume Rendering at Different Viewing Angles
    Nangles = 10
    for i in range(Nangles):
        print("Rendering Scene " + str(i + 1) + " of " + str(
            Nangles) + ".\n")

        # Camera Grid / Query Points -- rotate camera view
        angle = np.pi / 2 * i / Nangles
        N = 180
        c = np.linspace(-N / 2, N / 2, N)
        qx, qy, qz = np.meshgrid(c, c, c)
        qxR = qx
        qyR = qy * np.cos(angle) - qz * np.sin(angle)
        qzR = qy * np.sin(angle) + qz * np.cos(angle)
        qi = np.array([qxR.ravel(), qyR.ravel(), qzR.ravel()]).T

        # Interpolate onto Camera Grid
        camera_grid = interpn(points, datacube, qi, method="linear
            ").reshape((N, N, N))

        # Do Volume Rendering
        image = np.zeros((camera_grid.shape[1], camera_grid.shape
            [2], 3))
```

```
        for dataslice in camera_grid:
            r, g, b, a = transferFunction(np.log(dataslice))
            image[:, :, 0] = a * r + (1 - a) * image[:, :, 0]
            image[:, :, 1] = a * g + (1 - a) * image[:, :, 1]
            image[:, :, 2] = a * b + (1 - a) * image[:, :, 2]

        image = np.clip(image, 0.0, 1.0)

        # Plot Volume Rendering
        # plt.figure(figsize=(4, 4), dpi=80)

        # plt.imshow(image)
        # plt.axis("off")

        # # Save figure
        # plt.savefig(
        # "volumerender" + str(i) + ".png", dpi=240, bbox_inches="
            tight", pad_inches=0
        # )

    # Plot Simple Projection -- for Comparison
    # plt.figure(figsize=(4, 4), dpi=80)

    # plt.imshow(np.log(np.mean(datacube, 0)), cmap="viridis")
    # plt.clim(-5, 5)
    # plt.axis("off")

    # # Save figure
    # plt.savefig("projection.png", dpi=240, bbox_inches="tight",
        pad_inches=0)
    # plt.show()

    return 0



def run():

    # paths = ["data/data_128.hdf5", "data/datacube.hdf5", "data/
        data_512.hdf5", "data/data_1024.hdf5"]
    #paths = ["data/data_128.hdf5", "data/datacube.hdf5", "data/
        data_512.hdf5"]#, "data/data_1024.hdf5"]

    path = "data/datacube.hdf5"

    iterations = 10
```

```
    run_times = [0] * iterations

    for i in range(iterations):
        t1 = time.time()
        main(path)
        t2 = time.time()
        run_times[i] = t2-t1

    mean = statistics.mean(run_times)
    stdev = statistics.stdev(run_times)
    print(f"Average run time over {iterations} iterations: {mean
        }")
    print(f"Standard deviation over {iterations} iterations: {
        stdev}")


run()
```

Appendix F CuPy optimized code

```
    import numpy as np
import cupy as cp
import matplotlib.pyplot as plt
import h5py as h5
#from scipy.interpolate import interpn
from cupyx.scipy.interpolate import interpn
import time

def transferFunction(x):
    r = (
        1.0 * cp.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * cp.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * cp.exp(-((x - -3.0) ** 2) / 0.5)
    )
    g = (
        1.0 * cp.exp(-((x - 9.0) ** 2) / 1.0)
        + 1.0 * cp.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.1 * cp.exp(-((x - -3.0) ** 2) / 0.5)
    )
    b = (
        0.1 * cp.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * cp.exp(-((x - 3.0) ** 2) / 0.1)
        + 1.0 * cp.exp(-((x - -3.0) ** 2) / 0.5)
```

```
    )
    a = (
        0.6 * cp.exp(-((x - 9.0) ** 2) / 1.0)
        + 0.1 * cp.exp(-((x - 3.0) ** 2) / 0.1)
        + 0.01 * cp.exp(-((x - -3.0) ** 2) / 0.5)
    )

    return r, g, b, a

#@profile
def main(file):
    """Volume Rendering"""

    # Load Datacube
    f = h5.File(file, "r")
    datacube = cp.array(f["density"])

    # Datacube Grid
    Nx, Ny, Nz = datacube.shape
    x = cp.linspace(-Nx / 2, Nx / 2, Nx)
    y = cp.linspace(-Ny / 2, Ny / 2, Ny)
    z = cp.linspace(-Nz / 2, Nz / 2, Nz)
    points = (x, y, z)

    # Do Volume Rendering at Different Viewing Angles
    Nangles = 10
    pre = time.time()
    for i in range(Nangles):
        #print("Rendering Scene " + str(i + 1) + " of " + str(
            Nangles) + ".\n")



        # Camera Grid / Query Points -- rotate camera view
        angle = np.pi / 2 * i / Nangles
        N = 180
        c = cp.linspace(-N / 2, N / 2, N)
        qx, qy, qz = cp.meshgrid(c, c, c)
        qxR = qx
        qyR = qy * cp.cos(angle) - qz * cp.sin(angle)
        qzR = qy * cp.sin(angle) + qz * cp.cos(angle)
        qi = cp.array([qxR.ravel(), qyR.ravel(), qzR.ravel()]).T

        # Interpolate onto Camera Grid
        camera_grid = interpn(points, datacube, qi, method="linear
            ").reshape((N, N, N))
```

```
        # Do Volume Rendering
        image = cp.zeros((camera_grid.shape[1], camera_grid.shape
            [2], 3))

        for dataslice in camera_grid:
            r, g, b, a = transferFunction(cp.log(dataslice))
            image[:, :, 0] = a * r + (1 - a) * image[:, :, 0]
            image[:, :, 1] = a * g + (1 - a) * image[:, :, 1]
            image[:, :, 2] = a * b + (1 - a) * image[:, :, 2]

        cp.cuda.Stream.null.synchronize()

    post = time.time()

    return post - pre


if __name__ == "__main__":
    file = ["datacube.hdf5", "data_300.hdf5", "data_400.hdf5", "
        data_500.hdf5"]
    times = []

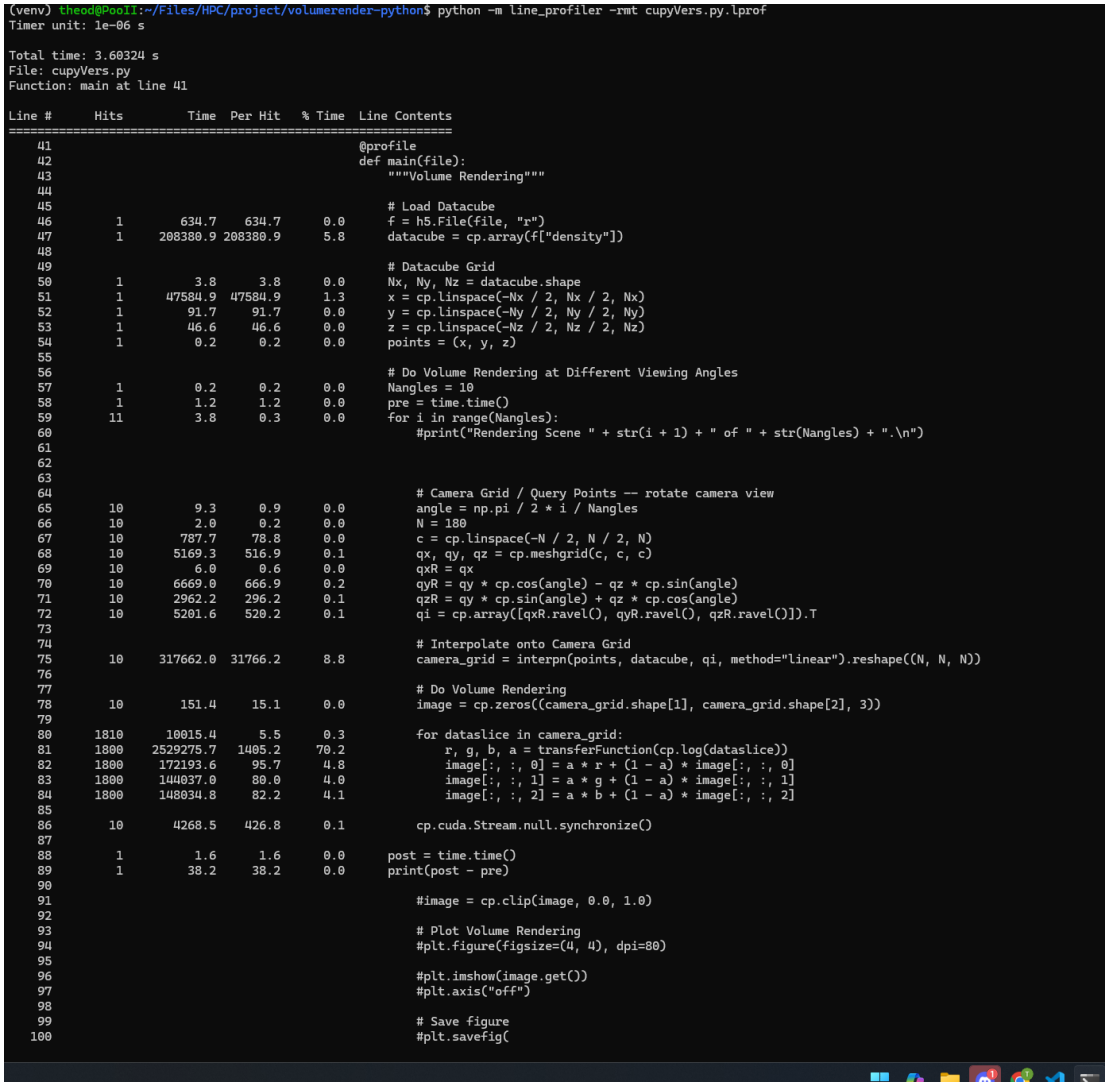
    for i in range(4):
        times.append([])
        for j in range(30):
            times[i].append(main(file[i]))
        print(f"{file[i]} done.")

    x = np.arange(30)

    for size, t in zip([256, 300, 400, 500], times):
        plt.plot(x, t, label=f"Size {size}")

    plt.xlabel("Iteration")
    plt.ylabel("Time (s)")
    plt.legend()
    plt.show()
```

Appendix G Line profiler for the cupy Code



```
(venv) theod@PooII:~/Files/HPC/project/volumerender-python$ python -m line_profiler -rmt cupyVers.py.lprof
Timer unit: 1e-06 s

Total time: 3.60324 s
File: cupyVers.py
Function: main at line 41

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    41                                           @profile
    42                                           def main(file):
    43                                               """Volume Rendering"""
    44
    45                                               # Load Datacube
    46         1        634.7    634.7      0.0      f = h5.File(file, "r")
    47         1     208380.9 208380.9      5.8      datacube = cp.array(f["density"])
    48
    49                                               # Datacube Grid
    50         1          3.8      3.8      0.0      Nx, Ny, Nz = datacube.shape
    51         1      47584.9  47584.9      1.3      x = cp.linspace(-Nx / 2, Nx / 2, Nx)
    52         1         91.7     91.7      0.0      y = cp.linspace(-Ny / 2, Ny / 2, Ny)
    53         1         46.6     46.6      0.0      z = cp.linspace(-Nz / 2, Nz / 2, Nz)
    54         1          0.2      0.2      0.0      points = (x, y, z)
    55
    56                                               # Do Volume Rendering at Different Viewing Angles
    57         1          0.2      0.2      0.0      Nangles = 10
    58         1          1.2      1.2      0.0      pre = time.time()
    59        11          3.8      0.3      0.0      for i in range(Nangles):
    60                                                   #print("Rendering Scene " + str(i + 1) + " of " + str(Nangles) + ".\n")
    61
    62
    63
    64                                                   # Camera Grid / Query Points -- rotate camera view
    65        10          9.3      0.9      0.0          angle = np.pi / 2 * i / Nangles
    66        10          2.0      0.2      0.0          N = 180
    67        10        787.7     78.8      0.0          c = cp.linspace(-N / 2, N / 2, N)
    68        10       5169.3    516.9      0.1          qx, qy, qz = cp.meshgrid(c, c, c)
    69        10          6.0      0.6      0.0          qxR = qx
    70        10       6669.0    666.9      0.2          qyR = qy * cp.cos(angle) - qz * cp.sin(angle)
    71        10       2962.2    296.2      0.1          qzR = qy * cp.sin(angle) + qz * cp.cos(angle)
    72        10       5201.6    520.2      0.1          qi = cp.array([qxR.ravel(), qyR.ravel(), qzR.ravel()]).T
    73
    74                                                   # Interpolate onto Camera Grid
    75        10     317662.0  31766.2      8.8          camera_grid = interpn(points, datacube, qi, method="linear").reshape((N, N, N))
    76
    77                                                   # Do Volume Rendering
    78        10        151.4     15.1      0.0          image = cp.zeros((camera_grid.shape[1], camera_grid.shape[2], 3))
    79
    80      1810      10015.4      5.5      0.3          for dataslice in camera_grid:
    81      1800    2529275.7   1405.2     70.2              r, g, b, a = transferFunction(cp.log(dataslice))
    82      1800     172193.6     95.7      4.8              image[:, :, 0] = a * r + (1 - a) * image[:, :, 0]
    83      1800     144037.0     80.0      4.0              image[:, :, 1] = a * g + (1 - a) * image[:, :, 1]
    84      1800     148034.8     82.2      4.1              image[:, :, 2] = a * b + (1 - a) * image[:, :, 2]
    85
    86        10       4268.5    426.8      0.1          cp.cuda.Stream.null.synchronize()
    87
    88         1          1.6      1.6      0.0      post = time.time()
    89         1         38.2     38.2      0.0      print(post - pre)
    90
    91                                                   #image = cp.clip(image, 0.0, 1.0)
    92
    93                                                   # Plot Volume Rendering
    94                                                   #plt.figure(figsize=(4, 4), dpi=80)
    95
    96                                                   #plt.imshow(image.get())
    97                                                   #plt.axis("off")
    98
    99                                                   # Save figure
   100                                                   #plt.savefig(
```

Figure 1: The line profiling of the cupy code